Q1 — Ideation: AI agents for Nexora's PMs/TPMs

Author: Srinivas Annam, AI PM/TPM · **Context:** Nexora Pvt. Ltd. runs several client projects at once with a small PM/TPM team (problem statement). · **Related:** BRD, Program charter

Three use cases where agents remove the most PM/TPM hours or prevent the most expensive surprises. Use cases 1 and 2 are built in Mira; use case 3 is the next candidate.

#	Use case	Pain it removes	Built?
1	Project plan and risk assessment generator	3–4 h per plan; inconsistent risk matrices	**Yes** — Planner + Risk Assessor agents
2	Weekly status, milestone tracking and stakeholder updates	1–2 h per weekly report; milestones missed with no early warning	**Yes** — Status Reporter + Stakeholder Update agents
3	Meeting notes to actions, decisions and RAID log	Notes scattered across meetings; action items lost; status reports rebuilt from memory	Next (proposed)

Use case 1 — Project plan and risk assessment generator

Pain point. Building a project plan from a high-level description takes **3–4 hours** of manual work per project, and risk assessments are inconsistent: some projects have detailed risk matrices, others have none. Senior PM time goes into formatting rather than judgement, and there is no standard format across the team.

Agentic solution. A router sends planning requests to a **Planner agent** and risk requests to a **Risk Assessor agent**. Each agent sees only its own slice of the project files, so it cannot borrow facts from elsewhere.

- **Planner:** phases, activities, milestones and deliverables taken from the timeline; goals quoted exactly from the description.
- **Risk Assessor:** a categorized matrix (category, risk, impact, likelihood, severity, mitigation) using only the project's risk register. It ranks risks by live task-board evidence, for example R09 (security) is first because T024 is blocked.
- **Router:** if the request is too vague ("We want to build a chatbot"), the router asks for scope, goals, timeline and team size instead of guessing.

Inputs → outputs

Inputs	Outputs
`project_description.txt` (scope, goals, approach, team)	Structured plan: objective and goals, scope, 8-phase table with dates and deliverables, key milestones, team, open questions

Inputs	Outputs
`project_timeline.csv` (phases, weeks, activities, milestones)	Risk matrix for all register risks, plus "what matters now" (top 3 by evidence)
`project_risks.csv` (category, risk, impact, mitigation)	Clarifying questions when the input is too thin
Task board (for risk evidence)	Every output marked "Draft for PM review", with its sources

Success metrics

Metric	Baseline	Target
Time to a reviewed project plan	3–4 h	< 45 min including PM review
Groundedness: milestones, goals and risks traceable to the source files	Not measured	≥ 95%; 0 invented risks or goals
Active projects with a current risk matrix	Inconsistent	100%

Knowledge base needed. The project description, the timeline (phases, weeks, deliverables) and the risk register. It also needs a short Nexora plan template (section order) and the task board for live risk evidence. Phase dates are computed in code from the project start date, not by the model.

Use case 2 — Weekly status, milestone tracking and stakeholder updates

Pain point. Weekly status reports take **1–2 hours** to compile from task boards and meeting notes. Milestones are missed because there are no proactive alerts, and issues surface only in retrospectives. Updates for leadership, engineering and the client are each rewritten by hand.

Agentic solution.

- **Code computes the facts first:** task counts, blocked, overdue and due-soon tasks, project health, per-sprint health and milestones due in the next 14 days. These are calculated from the board export for a set reporting date.
- **Status Reporter agent:** writes the weekly or sprint report from those facts, and answers tracking questions ("Which tasks are blocked or at risk?", "What milestones are coming up?", "Who owns T013?").
- **Stakeholder Update agent:** writes the same facts as an email for leadership, engineering or the client. Client emails leave out internal owner names and task IDs.
- **Not yet built:** a scheduled Monday run that posts at-risk milestones to Slack for PM approval. It is on the roadmap.

Inputs → outputs

Inputs	Outputs
Task board export (CSV: ID, name, status, owner, priority, sprint, due date, description)	Weekly or sprint status report: health and reason, counts, completed, in progress, blocked, overdue/due soon, upcoming milestones, next steps
`project_timeline.csv`	Tracking answers: blocked, overdue, due-soon tasks with owners and dates
Reporting date and audience (leadership / engineering / client)	Stakeholder email per audience; clarifying question when no task data is given

Success metrics

Metric	Baseline	Target
Time to a reviewed weekly status report	1–2 h	< 15 min
Count accuracy against the board export	Not measured	100%
Blocked tasks and at-risk milestones flagged before the retrospective	Found in retros	≥ 90%

Knowledge base needed. The task board export (one standard CSV format), the timeline for milestone dates, and the reporting date. It also needs the audience rules (what each audience may see) and a status-report template with a fixed section order.

Use case 3 — Meeting notes to actions, decisions and RAID log (next)

Pain point. Status reports are "compiled manually from task boards **and meeting notes**". Decisions and action items from standups, client calls and steering meetings live in personal notes. They are lost or re-asked, and risks raised in meetings never reach the risk register. With several projects per PM, this follow-up is what slips first.

Agentic solution. Three agents in sequence, then a PM approval step:

- **Extractor agent:** reads a meeting transcript or notes and pulls out decisions, action items (owner, due date) and new risks or issues. It quotes the source sentence for each.
- **Matcher agent:** links each item to an existing task ID or risk ID, or marks it as new, and flags conflicts. Example: "the notes say T024 is unblocked, but the board still says Blocked".
- **Drafter agent:** proposes RAID-log (risks, actions, issues, decisions) and task-board updates, plus a follow-up email to attendees.

Nothing is written to the board or sent until the PM approves.

Inputs → outputs

Inputs	Outputs
Meeting transcript or notes (text, or a Zoom/Meet transcript)	Decisions log and action items with owner, due date and the source quote
Current task board and risk register	Proposed RAID updates and task status changes, marked new or matched to existing IDs
Attendee list	Follow-up email draft; list of conflicts between the notes and the board

Success metrics

Metric	Baseline	Target
Action items captured with an owner and due date (vs. PM's manual list)	Not measured	≥ 90% recall
Time from meeting end to follow-up sent	Same day to never	< 30 min
Meeting-raised risks that reach the risk register within a week	Rarely	≥ 80%

Knowledge base needed. Meeting transcripts or notes, the current task board and risk register (to match IDs), the attendee list with roles, and the RAID template. It also needs a style guide for follow-up emails. Transcripts are client-confidential, so retention and redaction rules apply (see reflection §5).

Why build 1 and 2 first

- **They cover all five pains in the problem statement:** plan time, inconsistent risks, report time, missed milestones and no standard format.
- **The project already has all the data they need:** description, timeline, risk register and task board.
- **Their outputs can be checked against ground truth:** 23 tests, 151 checks.

Use case 3 needs meeting transcripts, which the project does not provide yet. It also adds a privacy review and write-back integration, so it follows once the pilot proves trust in use cases 1 and 2.